

Agregando "Modificar" a la Etapa 2-BD (con Filtro)

Objetivo: que al lado de cada botón **Eliminar** aparezca un botón **Modificar**. Al presionarlo, el formulario de `index.php` se cargue con los datos de esa persona, el título cambie a "Modificar Persona" y el botón pase de "Enviar" a "**Modificar**". Al guardar, los datos se actualicen en MySQL con un `UPDATE`.

Punto de partida: tenés la aplicación **Etapa 2-BD con Filtro** funcionando — ya guarda, lista, filtra por nombre y elimina. Le falta la letra **U** de **CRUD** (Update).

Carpeta del proyecto: `etapa2-BD-FiltroyUpdate/` (es una copia de `etapa2-BD-Filtro` con los cambios de esta guía).

0. ¿Qué es CRUD y qué nos falta?

Letra	Operación SQL	¿Ya la teníamos?	Archivo que la hace
Create	INSERT	Sí	<code>insertarPersona()</code> en <code>bd/gestionBaseDatos.php</code> + <code>procesando.php</code>
Read	SELECT	Sí (con filtro)	<code>obtenerPersonasConFiltro()</code> en <code>resultado.php</code>
Update	UPDATE	No — es lo que vamos a agregar	Nueva función <code>actualizarPersona()</code>
Delete	DELETE	Sí	<code>eliminarPersona()</code>

Un "Modificar" no es otra página nueva: **reutilizamos** `index.php` para dos usos:

- Sin `?id=` → formulario vacío → crea una persona nueva (como siempre).
- Con `?id=5` → formulario precargado → edita la persona 5.

1. Vista general: ¿qué vamos a tocar?

Solo **4 archivos**, con cambios muy puntuales:

Archivo	Qué agregamos
<code>bd/gestionBaseDatos.php</code>	Dos funciones nuevas: <code>obtenerPersonaPorId()</code> y <code>actualizarPersona()</code>
<code>resultado.php</code>	Un botón/link Modificar al lado de cada Eliminar
<code>index.php</code>	Lógica arriba para detectar <code>?id=</code> , precargar <code>value="..."</code> y cambiar el botón a "Modificar"
<code>procesando.php</code>	Un nuevo <code>if</code> que detecta <code>accion ===</code> <code>'actualizar'</code> y hace el <code>UPDATE</code>

`bd/script.sql` no cambia. La tabla `personas` sigue igual.

Flujo nuevo comparado con el de la Etapa 2:

```
ANTES (solo crear):
index.php (vacío) → POST → procesando.php (INSERT) → resultado.php

AHORA (crear o modificar):
resultado.php --click Modificar--> index.php?id=5 (precargado) --POST con
accion=actualizar--> procesando.php (UPDATE) → resultado.php
```

2. Paso 1 — La base de datos: dos funciones nuevas en `bd/gestionBaseDatos.php`

Abrí `bd/gestionBaseDatos.php` y al final del archivo, **justo antes de** `eliminarPersona()`, agregá estas dos funciones:

2.1 `obtenerPersonaPorId()` — traer UNA sola persona

```

<?php
/**
 * Obtiene UNA persona por su ID.
 * Se usa para precargar el formulario de edicion (index.php?id=X).
 *
 * fetch() trae UNA sola fila (a diferencia de fetchAll que trae todas).
 *
 * @param PDO $conexion Conexion a la base de datos
 * @param int $id ID de la persona
 * @return array|null La persona encontrada o null si no existe
 */
function obtenerPersonaPorId($conexion, $id) {
    $sql = "SELECT * FROM personas WHERE id = ?";
    $stmt = $conexion->prepare($sql);
    $stmt->execute([$id]);

    // fetch trae un solo registro como array asociativo
    return $stmt->fetch(PDO::FETCH_ASSOC);
}

```

Claves:

- `WHERE id = ?` — filtramos por clave primaria. Nunca traemos "todas" cuando solo queremos una.
- `$stmt->fetch()` — devuelve `['id'=>5, 'nombre'=>'Julián', ...]` o `false` si no existe. En `fetchAll` devolvía un array de arrays.
- `? + execute([$id])` — seguimos usando *prepared statements* para evitar inyección SQL (igual que en `eliminarPersona`).

2.2 actualizarPersona() — modificar una fila existente

```

<?php
/**
 * Actualiza los datos de una persona existente.
 *
 * UPDATE: modifica una fila que ya existe, identificada por su id.
 * SET columna = ? : cada ? se reemplaza por el valor nuevo.
 * WHERE id = ? : ¡importante! Sin el WHERE se modificarían TODAS las filas.
 *
 * @param PDO $conexion Conexion a la base de datos
 * @param int $id ID de la persona a modificar
 * @param array $datos Array asociativo con los datos nuevos
 * @return bool true si actualizo correctamente
 */
function actualizarPersona($conexion, $id, $datos) {
    $sql = "UPDATE personas
        SET nombre = ?, apellido = ?, dni = ?, cuit = ?,

```

```

    fecha_nacimiento = ?,
    email = ?, telefono = ?, direccion = ?, ciudad = ?,
    provincia = ?,
    codigo_postal = ?, observaciones = ?
    WHERE id = ?";

$stmt = $conexion->prepare($sql);
$resultado = $stmt->execute([
    $datos['nombre'],
    $datos['apellido'],
    $datos['dni'],
    $datos['cuit'],
    $datos['fecha_nacimiento'],
    $datos['email'],
    $datos['telefono'],
    $datos['direccion'],
    $datos['ciudad'],
    $datos['provincia'],
    $datos['codigo_postal'],
    $datos['observaciones'],
    $id // ¡El id va al final! Corresponde al ? del WHERE
]);

return $resultado;
}

```

Puntos para no equivocarte:

Concepto	Detalle
Orden de los?	Los 12 primeros? son las columnas del SET, el último ? es el WHERE id = ?. En execute([...]) el \$id tiene que ir último.
UPDATE sin WHERE	UPDATE personas SET nombre='Ana' sin WHERE le cambiaría el nombre a todas las personas. El WHERE id = ? asegura que solo cambia una.
insertarPersona vs actualizarPersona	INSERT crea una fila nueva (no necesita WHERE), UPDATE modifica una existente (sí necesita WHERE).

Dejá insertarPersona(), obtenerTodasLasPersonas(), obtenerPersonasConFiltro() y eliminarPersona() como estaban. No las borres.

3. Paso 2 — La lista: agregar el botón

"Modificar" en resultado.php

En `resultado.php`, dentro del `foreach ($personas as $persona)`, en la columna `<td>` de **Acciones** ya tenés el formulario de **Eliminar**. Ahora lo envolvemos en un `div` flex y agregamos un link **Modificar** antes:

```
<td>
  <div class="d-flex gap-1">
    <!--
      Boton MODIFICAR:
      - Es un simple link <a> con ?id=
      - Al hacer click, lleva a index.php?id=X
      - index.php detecta el id, busca la persona y precarga el
formulario
      - Usa GET porque es una consulta (no modifica nada aun)
    -->
    <a href="index.php?id=<?php echo $persona['id']; ?>" class="btn btn-
warning btn-sm">Modificar</a>

    <!-- Boton ELIMINAR (ya existente) -->
    <form action="procesando.php" method="POST" class="d-inline"
      onsubmit="return confirm('Seguro que deseas eliminar esta
persona?');">
      <input type="hidden" name="accion" value="eliminar">
      <input type="hidden" name="id" value="<?php echo $persona['id'];
?>">
      <button type="submit" class="btn btn-danger btn-
sm">Eliminar</button>
    </form>
  </div>
</td>
```

¿Por qué un `<a>` y no un `<form>`?

- **Modificar** solo *consulta* datos para mostrar el formulario, no cambia nada en la BD todavía. Por eso usa **GET** (`index.php?id=5`), igual que el filtro usa `?busqueda=...`. Queda en la URL y podés ver qué persona estás editando.
- **Eliminar** sí cambia la BD, por eso usa **POST** con campos ocultos y `confirm()`.

Visualmente, con Bootstrap `d-flex gap-1` los dos botones quedan uno al lado del otro (amarillo para Modificar, rojo para Eliminar).

4. Paso 3 — El formulario que sirve para

crear y para editar: `index.php`

Este es el cambio más grande pero sigue una idea simple: **el mismo archivo hace dos cosas.**

4.1 Bloque PHP al inicio (antes del `<!DOCTYPE html>`)

Reemplazá el inicio vacío por este bloque:

```
<?php
/*
    INDEX.PHP - Formulario de alta y edicion (ETAPA 2 con Filtro + Update)

    Este formulario sirve para DOS cosas:
    1. CREAR una persona nueva -> cuando se accede sin ?id= (ej: index.php)
    2. EDITAR una persona      -> cuando se accede con ?id= (ej:
index.php?id=5)
*/
require_once 'bd/gestionBaseDatos.php';

$conexion = obtenerConexion();

// Variables para el modo edicion
$persona = null;
$esEdicion = false;

// Si la URL trae ?id=, estamos editando una persona existente
if (isset($_GET['id'])) {
    $id = (int) $_GET['id'];
    $persona = obtenerPersonaPorId($conexion, $id);
    if ($persona) {
        $esEdicion = true;
    }
}
?>
```

Lectura línea por línea:

1. `isset($_GET['id'])` — ¿la URL vino como `index.php?id=5`? Si es `index.php` a secas, da `false` y todo queda en modo **crear**.
2. `(int) $_GET['id']` — convertimos a número entero por seguridad (si alguien escribe `?id=hola` queda 0).
3. `obtenerPersonaPorId($conexion, $id)` — buscamos esa persona en MySQL. Si no existe, `$persona` queda `false` y `$esEdicion` sigue `false`.

4.2 Título y encabezado dinámico

```
<title><?php echo $esEdicion ? 'Modificar Persona' : 'Formulario de
Contacto'; ?></title>
...
<h2 class="d-inline align-middle"><?php echo $esEdicion ? 'Modificar
Persona' : 'Formulario de Contacto'; ?></h2>
```

Usamos el **operador ternario** `condición ? valorSiTrue : valorSiFalse` (un `if` en una línea). Si estamos editando, el navegador muestra "Modificar Persona" en la pestaña y en el cartel azul.

4.3 Mensaje si el ID no existe

Justo después de `<div class="card-body">`, agregamos:

```
<?php if (isset($_GET['id']) && !$esEdicion): ?>
    <div class="alert alert-danger">La persona no existe o el ID es
    invalido.</div>
    <a href="resultado.php" class="btn btn-primary w-100 mb-3">Volver al
    listado</a>
<?php else: ?>
    <!-- ... el formulario va acá ... -->
<?php endif; ?>
```

Si alguien entra a `index.php?id=9999` y esa persona no existe, no mostramos un formulario vacío confuso sino un cartel claro.

4.4 Campos ocultos dentro del `<form>`

Dentro de `<form action="procesando.php" method="POST">`, bien arriba:

```
<?php if ($esEdicion): ?>
    <!-- Campos ocultos para el UPDATE: accion + id de la persona -->
    <!-- Sin estos campos, procesando.php no sabria que es una modificacion
    -->
    <input type="hidden" name="accion" value="actualizar">
    <input type="hidden" name="id" value="<?php echo $persona['id']; ?>">
<?php endif; ?>
```

- `type="hidden"` — no se ve, pero viaja en el `POST`. `procesando.php` lo leerá como `$_POST['accion'] === 'actualizar'`.
- En modo **crear** estos campos no existen, y `procesando.php` hará el `INSERT` normal.

4.5 Precargar cada input con `value="..."`

Cada campo pasa de:

```
<input type="text" id="nombre" name="nombre" required>
```

a:

```
<input type="text" id="nombre" name="nombre" required
      value="<?php echo $esEdicion ? htmlspecialchars($persona['nombre']) :
      ''; ?>">
```

Y para el `textarea` (que no usa `value` sino contenido interno):

```
<textarea id="observaciones" name="observaciones" rows="3"><?php echo
$esEdicion ? htmlspecialchars($persona['observaciones']) : ''; ?></textarea>
```

Tabla de cambios por campo:

Campo	Código precargado
nombre, apellido, dni, cuit, email, telefono, ciudad, provincia, codigo_postal, direccion	<code>value="<?php echo \$esEdicion ? htmlspecialchars(\$persona['campo']) : ''; ?>"</code>
fecha_nacimiento (type=date)	Igual, pero el formato debe ser <code>YYYY-MM-DD</code> como lo guarda MySQL
observaciones (textarea)	Contenido entre etiquetas, no atributo <code>value</code>

`htmlspecialchars()` es clave: si el nombre tenía una comilla (`D'Angelo`) o un `<`, no rompe el HTML.

4.6 Botón que cambia de texto y color

Reemplazá:

```
<button type="submit" class="btn btn-primary">Enviar</button>
```

por:

```
<button type="submit" class="btn <?php echo $esEdicion ? 'btn-warning' :  
'btn-primary'; ?>">  
    <?php echo $esEdicion ? 'Modificar' : 'Enviar'; ?>  
</button>
```

- Crear → botón azul **Enviar** (`btn-primary`).
- Editar → botón amarillo **Modificar** (`btn-warning`) para que se note la diferencia.

Opcional, debajo del botón en modo edición, agregamos un "Cancelar":

```
<?php if ($esEdicion): ?>  
<div class="d-grid mt-2">  
    <a href="index.php" class="btn btn-outline-secondary">Cancelar  
edicion</a>  
</div>  
<?php endif; ?>
```

Así el usuario puede volver al formulario vacío sin modificar nada.

5. Paso 4 — El cerebro: enseñarle a `procesando.php` a hacer UPDATE

`procesando.php` ya sabía hacer dos cosas: **eliminar** y **insertar**. Ahora le agregamos el medio, **actualizar**, entre ambas.

El orden de los `if` importa:

```

<?php
if ($_SERVER['REQUEST_METHOD'] === 'POST') {

    // CASO 1: ELIMINAR
    if (isset($_POST['accion']) && $_POST['accion'] === 'eliminar') {
        $id = (int) $_POST['id'];
        eliminarPersona($conexion, $id);
        header("Location: resultado.php");
        exit;
    }

    // CASO 2: ACTUALIZAR (¡NUEVO!)
    if (isset($_POST['accion']) && $_POST['accion'] === 'actualizar') {
        $id = (int) $_POST['id'];

        if (isset($_POST['nombre']) && isset($_POST['apellido']) &&
isset($_POST['dni'])) {
            $datos = [
                'nombre'           => trim($_POST['nombre']),
                'apellido'         => trim($_POST['apellido']),
                'dni'               => trim($_POST['dni']),
                'cuit'             => trim($_POST['cuit']),
                'fecha_nacimiento' => $_POST['fecha_nacimiento'],
                'email'            => trim($_POST['email']),
                'telefono'         => trim($_POST['telefono']),
                'direccion'        => trim($_POST['direccion']),
                'ciudad'           => trim($_POST['ciudad']),
                'provincia'        => trim($_POST['provincia']),
                'codigo_postal'    => trim($_POST['codigo_postal']),
                'observaciones'    => trim($_POST['observaciones']),
            ];

            // UPDATE en la base de datos
            actualizarPersona($conexion, $id, $datos);
        }

        header("Location: resultado.php");
        exit;
    }

    // CASO 3: INSERTAR (el que ya teníamos)
    if (isset($_POST['nombre']) && isset($_POST['apellido']) &&
isset($_POST['dni'])) {
        $datos = [ /* ... mismo array que arriba ... */ ];
        insertarPersona($conexion, $datos);
        header("Location: resultado.php");
        exit;
    } else {
        header("Location: index.php");
        exit;
    }
}

```

¿Por qué este orden?

1. Primero se pregunta si es **eliminar** (es el único que no necesita **nombre/apellido/dni**).
2. Después si es **actualizar** — también usa **accion** pero necesita los datos del formulario.
3. Al final, si no era ninguna acción especial, se asume que es un **alta** (solo campos normales).

Si pusieras el **INSERT** antes del **UPDATE**, un envío de modificación también cumpliría `isset($_POST['nombre'])` y haría un **INSERT** duplicado en vez de un **UPDATE**.

Siempre después de `header("Location: ...")` va `exit;` para que el script se detenga y no siga ejecutando los otros `if`.

6. Cómo probar que todo funciona

1. Levantá el servidor desde la carpeta nueva:

```
cd etapa2-BD-FiltroyUpdate
php -S localhost:8000
```

2. Flujo de **crear** (debe seguir igual):

- Entrá a `http://localhost:8000/index.php` → el formulario está vacío, dice "Formulario de Contacto", botón azul "Enviar".
- Cargá una persona nueva → te lleva a `resultado.php` y la ves en la tabla con los dos botones.

3. Flujo de **modificar**:

- En `resultado.php`, hacé click en **Modificar** en la fila de "María Perez".
- La URL debe cambiar a `http://localhost:8000/index.php?id=2` → el formulario aparece con todos los campos ya llenos, título "Modificar Persona" y botón amarillo "Modificar".
- Cambiá el teléfono o la ciudad y apretá **Modificar** → volvé a `resultado.php` y ves la fila actualizada (sin que se duplique).
- Recargá la página y usá el **filtro** (`?busqueda=Mar`) para confirmar que el filtro sigue funcionando después del **UPDATE**.

4. Casos de error a probar:

Qué hacés	Qué esperás
<code>index.php?id=9999</code> (id que no existe)	Cartel rojo "La persona no existe" en vez de formulario vacío
Entrar a <code>index.php?id=hola</code>	<code>(int)</code> lo convierte a <code>0</code> , muestra el mismo cartel de error
Click en "Cancelar edicion"	Vuelve a <code>index.php</code> vacío (modo crear)
Click en "Ver Personas" después de modificar	Vuelve al listado filtrado o completo según como entraste

5. Bonus: abrí las herramientas del navegador (F12 → Network) y mirá:

- Click en **Modificar** → petición **GET** a `index.php?id=2` (no cambia la BD).
- Submit del formulario editado → petición **POST** a `procesando.php` con `accion=actualizar&id=2` en el payload.

7. Resumen visual: qué cambió en cada archivo

```

bd/gestionBaseDatos.php
+ obtenerPersonaPorId($conexion, $id) → SELECT * WHERE id = ?
+ actualizarPersona($conexion,$id,$datos) → UPDATE ... SET ... WHERE id =
?

resultado.php
+ <a href="index.php?id=...">Modificar</a> (GET, al lado de Eliminar)
(mantiene el buscador por GET y el mensaje diferenciado)

index.php
+ Bloque PHP arriba: detecta ?id, llama a obtenerPersonaPorId
+ $esEdicion ? : para título, values, hidden inputs y botón
+ <input hidden accion=actualizar> + <input hidden id>
+ value="htmlspecialchars($persona['campo'])" en cada input

procesando.php
+ if (accion === 'actualizar') → arma $datos y llama a actualizarPersona()
(mantiene eliminar e insertar)

```

Con esto completaste el **CRUD** en una sola tabla sin claves foráneas, que es justo el paso previo a la Etapa 3 (donde aparecen `ciudades` y `ciudad_id`).

¡Con el filtro y el modificar ya tenés una aplicación web completa con base de datos, lista para entender por qué en la Etapa 3 se separa **ciudad** a otra tabla!